C++ Functions Lab Practice

Lab 6 — Functions: Parameters & Return Values

Parameters + Return | Parameters + No Return | Logic Building

Note: Questions 1 OR 2 in each part include reference code. Write your own complete solutions for the rest of the questions.

Restriction: Use only int, float, double, char, bool. No strings, arrays, or pointers.

Objectives

1. Pass simple variables as parameters to functions.
2. Write functions that receive parameters and return a computed result.
3. Write void functions that receive parameters and perform actions (print, calculate).
4. Build multi-step logic: conditions, loops, and arithmetic inside function bodies.
5. Write function prototypes when definitions appear after main().
6. Call functions correctly from main() using the right arguments.

Part 1: Functions — With Parameters, With Return Value

These functions **receive one or more parameters** and **return a computed result** back to the caller. The caller stores or prints that result.

Q1. Define square(int n) that returns n * n as an int. Call it from main() with value 6 and print the result. (Code provided as reference.)

Expected Output:

```
Square of 6 = 36
```

C++ Code:

```
int square(int n) {
    return n * n;
}

int main() {
    int result = square(6);
    cout << "Square of 6 = " << result << endl;
    return 0;
}
```

Q2. Define celsiusToFahrenheit(float c) that converts Celsius to Fahrenheit using F = c * 9.0/5.0 + 32 and returns a float. Call it from main() with 37.0. (Code provided as reference.)

Expected Output:

```
37.0 C = 98.60 F
```

C++ Code:

```
float celsiusToFahrenheit(float c) {
    return c * 9.0 / 5.0 + 32;
}

int main() {
    float f = celsiusToFahrenheit(37.0);
    cout << "37.0 C = " << f << " F" << endl;
    return 0;
}
```

Q3. Define add(int a, int b), subtract(int a, int b), and multiply(int a, int b) — each returns an int result. Call all three from main() with values 12 and 4 and print each result.

Expected Output:

```
12 + 4 = 16
12 - 4 =  8
12 x 4 = 48
```

Q4. Define isEven(int n) that returns true if n is even, false otherwise. Define isPositive(int n) that returns true if n > 0. Call both from main() with n = -6 and print 'Yes' or 'No' for each.

Expected Output:

```
-6 is Even    : Yes
-6 is Positive: No
```

Q5. Define calcSimpleInterest(float principal, float rate, float time) that returns SI = (principal * rate * time) / 100.0. Also define calcTotalAmount(float principal, float si) that returns principal + si. Call both from main() with principal=5000, rate=8.0, time=3.

Expected Output:

```
Simple Interest = 1200.00
Total Amount    = 6200.00
```

Q6. Define maxOfThree(int a, int b, int c) that returns the largest of three integers. Call it from main() with (14, 37, 22) and (99, 99, 50) — handle the tie case correctly.

Expected Output:

```
Max of 14, 37, 22 = 37
Max of 99, 99, 50 = 99
```

Q7. Define getGradeLetter(int marks) that returns a char grade: 'A' (>=80), 'B' (>=70), 'C' (>=60), 'D' (>=50), 'F' (below 50). Define calcPercentage(int obtained, int total) that returns float percentage. In main(), use both functions with obtained=74 and total=100, and also use isEven() from Q5 to check if marks are even.

Expected Output:

```
Obtained    = 74 / 100
Percentage  = 74.00
Grade       = C
Marks Even  : Yes
```

Parameters + Return — Key Rules

Situation	Syntax / Rule
define with param + return	`int funcName(int a, float b) { return ...; }`

call and store result	`int r = funcName(5, 3.2);`
return type must match value	`float f(int x) { return x * 1.5; }`
bool return with condition	`bool check(int x) { return x >= 50; }`
char return with if-chain	`char grade(int m) { if(m>=80) return 'A'; ... }`
call result directly in cout	`cout << funcName(a, b) << endl;`

Part 2: Functions — With Parameters, No Return (void)

These functions **receive parameters** but **return nothing (void)**. They use the parameters to print output or perform an action directly.

Q1. Define printTemperatureStatus(float temp) that prints 'Hot' if temp > 35, 'Warm' if > 20, or 'Cold' otherwise. Call it three times from main() with 40.0, 25.0, and 10.0. (Code provided as reference.)

Expected Output:

```
40.00 C -> Hot
25.00 C -> Warm
10.00 C -> Cold
```

C++ Code:

```
void printTemperatureStatus(float temp) {
    cout << temp << " C -> ";
    if      (temp > 35) cout << "Hot"  << endl;
    else if (temp > 20) cout << "Warm" << endl;
    else                cout << "Cold" << endl;
}

int main() {
    printTemperatureStatus(40.0);
    printTemperatureStatus(25.0);
    printTemperatureStatus(10.0);
    return 0;
}
```

Q2. Define printMultTable(int n) that prints the complete multiplication table of n from 1 to 10 using a loop. Call it from main() with n = 7.

Expected Output:

```
7 x  1 =  7
7 x  2 = 14
7 x  3 = 21
...  ...
7 x 10 = 70
```

Q3. Define printBill(int qty, float unitPrice) that prints a formatted bill showing quantity, unit price, and total (qty * unitPrice). Call it from main() with qty = 4 and unitPrice = 75.50.

Expected Output:

```
Quantity   :   4
Unit Price : 75.50
Total Bill : 302.00
```

Q4. Define printStudentResult(int marks, int total) that prints marks, total, percentage, and Pass/Fail (pass if percentage >= 50). Call it twice: once with 45, 100 and once with 78, 100.

Expected Output:

```
Marks      : 45 / 100
Percentage : 45.00
Result     : Fail
-------------------
Marks      : 78 / 100
Percentage : 78.00
Result     : Pass
```

Q5. Define printEvenOddTable(int limit) that prints a table of numbers from 1 to limit showing whether each is Even or Odd. Call it from main() with limit = 8.

Expected Output:

```
Num   Type
----------
1     Odd
2     Even
3     Odd
4     Even
5     Odd
6     Even
7     Odd
8     Even
```

Q6. Define printFizzBuzz(int limit) that prints numbers from 1 to limit but: prints 'Fizz' if divisible by 3, 'Buzz' if divisible by 5, 'FizzBuzz' if divisible by both, otherwise the number. Call it from main() with limit = 15.

Expected Output:

```
1  2  Fizz  4  Buzz  Fizz  7  8  Fizz  Buzz  11  Fizz  13  14  FizzBuzz
```

void with Parameters — Key Rules

Situation	Syntax / Rule
define void with parameters	`void funcName(int x, float y) { ... }`
call with arguments	`funcName(5, 3.2);`
use parameter inside body	`cout << x * 2 << endl;`
prototype when defined after	`void funcName(int x);  // above main`
multiple calls with diff args	`funcName(10); funcName(20);`
nested loops for patterns	`for(i) { for(j) { cout << '*'; } }`

Part 3: Logic Building — Parameters + Return Value

These questions require **multi-step logic inside the function body**: conditions, loops, counters, and arithmetic — all combined with parameters and a return value.

Q1. Define power(int base, int exp) that returns base raised to the power exp using a loop (do NOT use pow()). Call it from main() with base=2, exp=8. (Code provided as reference.)

Expected Output:

```
2 ^ 8 = 256
```

Hint: Start result = 1, then multiply result by base, exp times, using a for loop.

C++ Code:

```
int power(int base, int exp) {
    int result = 1;
    for (int i = 0; i < exp; i++) {
        result *= base;
    }
    return result;
}

int main() {
    cout << "2 ^ 8 = " << power(2, 8) << endl;
    return 0;
}
```

Q2. Define sumDigits(int n) that returns the sum of all digits of a positive integer. (e.g. 253 → 2+5+3 = 10). Call it from main() with 253. (Code provided as reference.)

Expected Output:

```
Sum of digits of 253 = 10
```

Hint: Use % 10 to extract the last digit, then / 10 to remove it. Repeat with a loop until n == 0.

C++ Code:

```
int sumDigits(int n) {
    int sum = 0;
    while (n != 0) {
        sum += n % 10;   // extract last digit
        n   /= 10;        // remove last digit
    }
    return sum;
}

int main() {
    cout << "Sum of digits of 253 = " << sumDigits(253) << endl;
    return 0;
}
```

Q3. Define countDigits(int n) that returns the number of digits in a positive integer (e.g. 4572 → 4 digits). Call it from main() with 4572 and 99.

Expected Output:

```
4572 has 4 digits
99   has 2 digits
```

Hint: Repeatedly divide n by 10 and count each step until n becomes 0.

Q4. Define isPrime(int n) that returns true if n is a prime number, false otherwise. Call it from main() with 7, 12, and 17 and print 'Prime' or 'Not Prime'.

Expected Output:

```
7  -> Prime
12 -> Not Prime
17 -> Prime
```

Hint: A number is prime if no integer from 2 to n-1 divides it evenly. Use a for loop and the % operator.

Q5. Define factorial(int n) that returns n! (n factorial) using a loop. Call it from main() with 5 and 7. Also call sumDigits() from Part 3 Q1 to print the digit sum of 7! as well.

Expected Output:

```
5! = 120
7! = 5040
Sum of digits of 5040 = 9
```

Hint: factorial(0) = 1. For n > 0: multiply 1 * 2 * 3 * ... * n using a for loop.

Q6. Define reverseNumber(int n) that returns the integer with its digits reversed (e.g. 1234 → 4321). Call it from main() with 1234 and 900.

Expected Output:

```
Reverse of 1234 = 4321
Reverse of 900  =    9
```

Hint: Extract the last digit using % 10, build reversed = reversed * 10 + digit, then remove the last digit with / 10.

Q7. Define isPerfect(int n) that returns true if n is a perfect number (a number equal to the sum of its proper divisors). Call it with 6, 28, and 12.

Expected Output:

```
6  -> Perfect
28 -> Perfect
12 -> Not Perfect
```

Hint: Proper divisors of n are all integers from 1 to n-1 that divide n with no remainder. Sum them with a loop and compare to n.

Q8. Define gcd(int a, int b) that returns the Greatest Common Divisor of two integers using the Euclidean algorithm. Then define lcm(int a, int b) that uses gcd() to compute LCM = (a * b) / gcd(a, b). Call both from main() with a=12, b=18.

Expected Output:

```
GCD of 12 and 18 = 6
LCM of 12 and 18 = 36
```

Euclidean hint: while b != 0 { int t = b; b = a % b; a = t; } then return a.

Logic Building — Parameters + Return: Key Patterns

Situation	Syntax / Rule

extract last digit of n	`digit = n % 10;  n /= 10;`
loop until n becomes 0	`while (n != 0) { ... n /= 10; }`
accumulate product in loop	`int r = 1; for(i=1;i<=n;i++) r *= i;`
build reversed number	`rev = rev * 10 + n % 10;  n /= 10;`
check divisibility	`if (n % i == 0)  // i divides n`
call one function inside another	`int l = (a * b) / gcd(a, b);`

— Excellent Work! —